
ProjectName – Java

Rapport final

Étudiant1, Étudiant2, Étudiant3, Étudiant4, Étudiant5

8 avril 2026

Table des matières

1	Historique et Nature du Sujet	3
1.1	Historique du sujet	3
1.2	Nature et enjeux du jeu	3
1.3	Complexite du jeu et defis algorithmiques	3
1.4	Existant algorithmique	4
2	Architecture du projet	4
2.1	Decoupage en modules	4
2.2	Dependances et flux principaux	5
2.3	Principales interfaces et points d’extension	5
2.4	Diagramme UML global (capture de reference)	5
3	L’Algorithme Minimax :	5
4	L’Algorithme Expectiminimax :	6
5	Comparaison des deux Algorithmes	7
6	Le Machine Learning :	8
7	DAWG et GADDAG	8
I	Spécifications Étendues	12
8	F21 — Sauvegarde et restauration d’une partie	12

1 Historique et Nature du Sujet

1.1 Historique du sujet

En 1931, l'architecte au chômage Alfred Mosher Butts decide de creer un jeu combinant chance et reflexion. En analysant minutieusement la frequence des lettres a la une du *New York Times*, il elabore un systeme de valeurs specifiques pour chaque jeton, methode qui reste inchangee dans les editions modernes [1]. Initialement baptise *Lexiko*, puis *Criss-Cross Words*, le jeu essuie les refus de nombreux fabricants avant qu'un homme d'affaires, James Brunot, ne rachete les droits en 1948 pour lui donner son nom definitif : Scrabble [2].

Le succes mondial du jeu doit beaucoup a un coup de chance marketing. Dans les annees 1950, Jack Straus, president du grand magasin Macy's a New York, decouvre le jeu pendant ses vacances et s'etonne de ne pas le trouver dans ses rayons. Il passe une commande massive, propulsant le Scrabble au rang de phenomene culturel [2]. Aujourd'hui, avec plus de 150 millions d'exemplaires vendus dans 121 pays et des competitions internationales organisees par la World English-Language Scrabble Players Association (WESPA) et la Federation Internationale de Scrabble Francophone (FISF), il est devenu le jeu de lettres le plus populaire de la planete [3].

Des les annees 1980 et 1990, les premiers logiciels de jeu contre l'ordinateur font leur apparition. L'explosion des plateformes en ligne, notamment l'Internet Scrabble Club (ISC), puis des applications mobiles comme *Scrabble Go* ont elargi considerablement la communaute de joueurs. Au-dela du divertissement, l'informatique a revolutionne la competition de haut niveau grace a des outils d'analyse strategique comme Quackle [4], devenu la reference mondiale pour l'entrainement des joueurs experts. Aujourd'hui, de nombreux projets open source et applications multiplateformes temoignent de la vitalite de l'ecosysteme logiciel autour du Scrabble.

Sources : [1] Butts, A.M. (1931), archives de conception du jeu. [2] Fatsis, S. (2001), *Word Freak*, Houghton Mifflin. [3] FISF, Regles officielles du Scrabble francophone, edition 2023. [4] Sheppard, B. et al., *Quackle Scrabble AI*, Computer Scrabble Project, 2006.

1.2 Nature et enjeux du jeu

Le Scrabble est un jeu de lettres a information imparfaite (les tirages adverses sont inconnus) et stochastique (soumis a l'alea de la pioche). L'objectif est de maximiser son score sur une grille standard de 15×15 cases en y plaçant des mots extraits d'un dictionnaire de reference (ODS en francais, TWL/SOWPODS en anglais).

Le jeu repose sur trois piliers fondamentaux :

1. **Logique spatiale** Tout mot pose doit respecter deux contraintes simultanees : la connexite (chaque nouveau mot doit s'appuyer sur au moins une lettre deja presente sur le plateau, et toutes les lettres posees en un meme tour doivent etre alignees et contigues) et l'orthogonalite (toute collision avec des lettres adjacentes doit former un mot valide dans le dictionnaire de reference).

Exemple : si le mot FEU est pose horizontalement, un joueur peut placer la lettre R au-dessus du E pour former RE verticalement, a condition de poser simultanement un mot horizontal valide passant par ce R, comme RATS.

2. **Logique mathematique** Le score d'un coup se calcule en additionnant la valeur de chaque lettre posee, en appliquant d'abord les multiplicateurs de lettres (case lettre compte double/triple), puis les multiplicateurs de mots (case mot compte double/triple). Un joueur qui utilise ses 7 lettres en un seul coup remporte un bonus de 50 points, appele Scrabble.

Exemple : au premier tour, un joueur pose AXE avec le X sur la case centrale H8 (case mot $\times 2$). La valeur brute est $A(1) + X(8) + E(1) = 10$, doublee a 20 points. Si la case centrale etait une case lettre $\times 2$ sous le X, on calculerait $A(1) + X(16) + E(1) = 18$, puis on appliquerait le multiplicateur de mot.

3. **Logique linguistique** Chaque mot pose, y compris les mots formes par collision, doit appartenir au dictionnaire de reference choisi pour la partie. Tout mot invalide entraine l'annulation du coup et, selon les regles, une penalite de score.

Deroulement d'une partie La partie debute obligatoirement par un mot d'au moins deux lettres couvrant la case centrale (H8), dont le score est automatiquement double. A chaque tour, le joueur peut : poser un mot, echanger tout ou partie de ses jetons contre des lettres de la pioche (si le sac contient au moins 7 lettres), ou passer son tour. La partie s'acheve lorsque le sac est vide et qu'un joueur a epuise son chevalet, ou apres trois tours consecutifs sans score. Les lettres restant sur les chevalets sont deduites du score de chaque joueur, et leur valeur totale est ajoutee au score du joueur ayant vide son chevalet en premier.

Exemple de fin de partie : le joueur A termine avec 300 points. Le joueur B possede encore un W (10 pts) et un X (10 pts) : son score final est 280 pts, et le score de A monte a 320 pts.

1.3 Complexite du jeu et defis algorithmiques

Il est essentiel de distinguer la complexite intrinseque du jeu de celle des algorithmes qui cherchent a le resoudre.

Espace des etats Le nombre de positions legales sur un plateau 15×15 est estime a environ 10^{30} . Si ce chiffre reste inferieur a celui des echecs ($\sim 10^{43}$), l'incertitude propre au Scrabble, due aux tirages caches de l'adversaire et a l'alea de la pioche, en fait un probleme fondamentalement different : il releve de la theorie des jeux a information imparfaite, contrairement aux echecs ou aux dames.

Explosion combinatoire a chaque coup Meme pour un seul tour, la generation de tous les coups legaux est un probleme non trivial. Pour un chevalet de 7 lettres et un dictionnaire de type ODS ($\sim 380\,000$ mots), il faut explorer toutes les combinaisons de lettres (sous-ensembles du chevalet), toutes les positions et orientations possibles sur la grille, et verifier simultanement la validite de tous les mots formes par collision. Le nombre de placements theoriques au premier tour est deja de l'ordre de plusieurs millions [5].

La qualite du reliquat Un des defis strategiques majeurs, et algorithmiques, est que maximiser le score immediat n'est pas toujours optimal. Un bon moteur de jeu doit evaluer la qualite du reliquat (les lettres conservees apres le coup), car des lettres polyvalentes comme les voyelles courantes ou le S offrent de meilleures perspectives pour les tours suivants. A l'inverse, conserver des lettres difficiles comme le W ou le K penalise les coups futurs.

La gestion des cases bonus Un autre defi crucial est l'obstruction : ouvrir une case bonus (mot $\times 3$ ou lettre $\times 3$ en bord de grille) pour l'adversaire peut lui offrir un avantage considerable. L'IA doit donc anticiper non seulement son propre score, mais aussi les opportunités qu'elle laisse a son adversaire.

Conclusion sur la difficulte Ces contraintes font du Scrabble un probleme d'optimisation combinatoire sous incertitude, a la frontiere entre la recherche arborescente, la linguistique computationnelle et la theorie des jeux stochastiques.

Source : [5] Appel, A.W. & Jacobson, G.J. (1988), *The World's Fastest Scrabble Program*, *Communications of the ACM*.

1.4 Existant algorithmique

L'etat de l'art repose sur des travaux academiques fondateurs et des logiciels eprouves.

Structures de donnees pour le dictionnaire La generation rapide de coups legaux repose sur des automates finis representant le dictionnaire :

- le DAWG (*Directed Acyclic Word Graph*), propose par Appel & Jacobson (1988) [5], compresse le dictionnaire en un graphe oriente acyclique permettant une verification en $O(n)$ de la validite d'un mot;
- le GADDAG, introduit par Steven Gordon (1994) [6], est une extension permettant de generer directement tous les mots pouvant s'ancrer sur une lettre donnee du plateau, accelerant considerablement la recherche des coups legaux.

Logiciels de reference Quackle [4] est le standard academique et competitif pour l'analyse strategique. Il utilise des simulations de type Monte-Carlo pour estimer l'esperance de gain de chaque coup sur plusieurs tours a venir, en simulant des milliers de parties possibles.

Elise et Maven sont d'autres moteurs historiques ayant contribue a l'etablissement des meilleures pratiques en IA Scrabble.

Plateformes de jeu en ligne L'ISC (*Internet Scrabble Club*) demeure la reference pour la pratique competitive, garantissant une application stricte des regles internationales. Des plateformes plus recentes comme Woogles.io proposent egalement des outils d'analyse integres.

Open source De nombreux projets open source (notamment en Python, Java et C++) sont disponibles sur des plateformes comme GitHub, rendant accessible l'implementation d'un moteur de jeu a partir des algorithmes decrits dans la litterature.

Source : [6] Gordon, S. (1994), *A Faster Scrabble Move Generation Algorithm*, *Software—Practice and Experience*.

2 Architecture du projet

2.1 Decoupage en modules

Le code est organise en couches fonctionnelles relativement bien separees :

- **Point d'entree** : `fr.ubordeaux.scrabble.App` initialise la configuration, la langue, puis lance le mode CLI ou GUI.
- **Couche controleur** : `controller` orchestre la logique applicative (demarrage de partie, traitement des commandes, enchainement des tours, integration reseau).
- **Couche metier** : `model.core` contient les objets de domaine (`Game`, `Board`, `Move`, `Scoring`, etc.) et les regles de jeu.
- **IA** : `model.ai` regroupe les strategies automatiques (Minimax, Expectiminimax, agent ML) et leur evaluation.
- **Dictionnaire** : `model.dictionary` fournit les structures de recherche lexicales (Trie, DAWG, GADDAG) utilisees pour la validation et la generation de coups.

- **Vue** : `view` expose l'interface utilisateur via `UserInterface`, avec implementations CLI et JavaFX.
- **Persistance** : `model.savefiles` gere la sauvegarde et la restauration de partie.
- **Reseau** : `model.network` prend en charge la decouverte, les echanges client/serveur et la synchronisation.

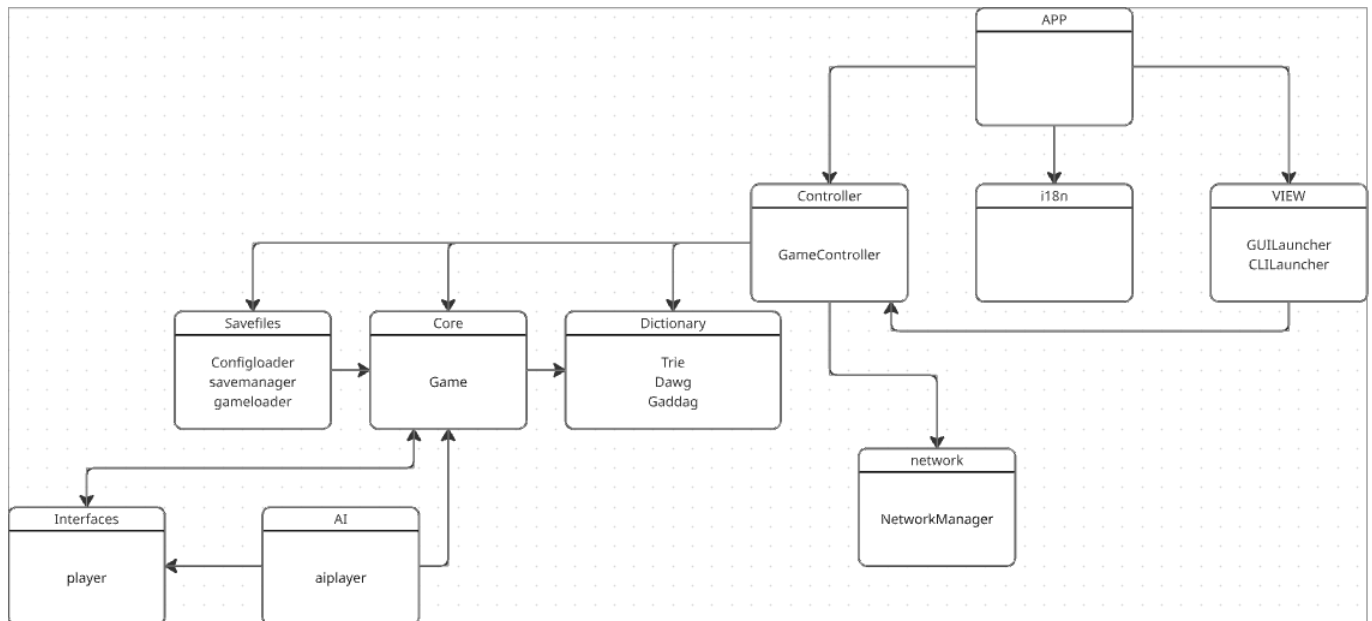


FIGURE 1 – Capture UML globale de l'architecture du projet (source : capture d'écran du diagramme UML).

2.2 Dependances et flux principaux

Le flux d'exécution principal suit un schema en couches :

1. **App** charge la configuration et le contexte d'exécution.
2. Le **GameController** instancie et pilote la partie via **Game**.
3. Le controleur s'appuie sur les services metier : dictionnaire, generation de coups, score, IA, sauvegarde.
4. La couche **view** affiche l'état du jeu et traduit les actions utilisateur en commandes.

Les dependances sont majoritairement descendantes (vue → controleur → modele), ce qui facilite la testabilite et limite le couplage entre interface et regles metier.

2.3 Principales interfaces et points d'extension

Les interfaces suivantes structurent l'evolutivite du projet :

- **Player** : abstraction commune des joueurs humains et IA, permettant de faire evoluer les strategies sans modifier le coeur du moteur.
- **UserInterface** : contrat commun des interfaces CLI/GUI, utile pour separer logique de presentation et logique metier.
- **NetworkObserver** : mecanisme d'observation pour propager les evenements reseau vers les composants interesses.

Cette organisation rend possible l'ajout de nouvelles interfaces (par exemple une vue web) ou de nouvelles IA avec un impact limite sur les autres modules.

3 L'Algorithme Minimax :

Conçu pour des jeux à information parfaite et à somme nulle comme les Échecs, l'algorithme Minimax modélise l'arbre des coups possibles en alternant un tour où l'IA cherche à maximiser son score, et un tour où elle simule un adversaire cherchant à minimiser l'avantage de l'IA.

Pour adapter cet algorithme au Scrabble (où le chevalet adverse est inconnu), notre implémentation déduit d'abord les tuiles restantes en soustrayant du jeu complet les lettres déjà posées sur le plateau et celles présentes dans le chevalet de l'IA. Ensuite, le solveur utilise une méthode de Monte-Carlo : il simule l'adversaire en tirant aléatoirement 200 chevalets virtuels de 7 lettres parmi ces tuiles restantes. Le Minimax évalue alors le pire scénario possible : parmi ces 200 simulations, il retient le coup adverse qui marque le plus de points, et le soustrait du score du coup envisagé par l'IA.

L'impact de l'heuristique

Au Scrabble, évaluer un coup uniquement par les points qu'il rapporte sur le plateau n'est pas le plus pertinent. L'algorithme intègre donc une fonction heuristique d'évaluation qui note la qualité des lettres conservées sur le chevalet après un coup. Cette heuristique donne des bonus et malus en fonction de :

- Un joker conservé rapporte un bonus massif de +15.0 points.
- La lettre 'S' rapporte +8.0 points.
- Les lettres difficiles (J, K, Q, X, Z) appliquent un malus de -6.0 points.
- L'équilibre voyelles/consonnes est récompensé (+5.0 points) tandis qu'un chevalet exclusivement composé de voyelles ou de consonnes est lourdement pénalisé (-12.0 points).

Complexité et coût

Dans notre implémentation de Monte-Carlo, l'algorithme ne disposant pas d'élagage Alpha-Beta, la complexité temporelle est strictement identique à celle de l'Expectiminimax, soit $O(B \times N \times B)$ pour une anticipation d'un tour de l'adversaire (où B est le facteur de branchement et $N = 200$ le nombre de tirages simulés). La complexité reste modérée car le parcours se fait en profondeur d'abord. Le coût est massif puisqu'il doit évaluer exactement les 200 scénarios complets. L'exécution est donc bridée par une limite de temps stricte (`timeLimits`), l'empêchant de calculer sur plusieurs tours d'anticipation.

Performances

La performance défensive est excellente. En s'attendant systématiquement au pire, l'IA verrouille le plateau et évite d'ouvrir des cases multiplicatrices dangereuses. Mais ce pessimisme absolu la rend parfois trop peu entreprenante sur le plan offensif. De plus, puisqu'elle paie le même coût que l'Expectiminimax sans bénéficier d'une prise de décision nuancée, cette implémentation se révèle, à temps de calcul égal, moins optimale que l'approche probabiliste.

4 L'Algorithme Expectiminimax :

L'Expectiminimax est l'évolution du Minimax pour les jeux intégrant du hasard. Il introduit un nouveau type de nœud dans l'arbre de recherche : les "nœuds de chance" (chance nodes). Au lieu de considérer que l'adversaire jouera le pire coup absolu, l'algorithme calcule l'espérance mathématique de tous les coups possibles en fonction de leur probabilité.

Dans notre code, la méthode `expectiminimax()` utilise la même base de simulation que le Minimax (les 200 tirages aléatoires de chevauxets adverses). Cependant, au lieu d'extraire la valeur maximale de ces simulations, l'algorithme calcule la moyenne des meilleurs scores adverses obtenus sur ces 200 échantillons. Il évalue ainsi un coup en soustrayant cette "réponse moyenne attendue" du score du coup de l'IA.

L'impact de l'heuristique

L'Expectiminimax ne se contente pas de calculer mathématiquement les risques sur le plateau. Pour juger de la force réelle d'un coup, l'IA additionne deux éléments concrets :

- **Le risque sur le plateau (L'algorithme) :** L'IA vérifie qu'elle ne laisse pas d'opportunités trop faciles et dangereuses à l'adversaire (comme ouvrir l'accès à une case "Mot Compte Triple").
- **La qualité de sa main (L'heuristique) :** L'IA évalue les lettres qu'elle garde sur son chevalet pour son prochain tour.

C'est le compromis entre ces deux visions qui rend l'IA véritablement intelligente. Par exemple, si un coup sécurise parfaitement le plateau mais oblige l'IA à conserver une main catastrophique, l'heuristique va agir comme un garde-fou et lourdement pénaliser ce choix. L'IA cherche donc toujours l'équilibre parfait entre "limiter la casse sur le plateau" et "garder de bonnes lettres pour l'avenir".

Complexité et coût

La complexité est de $O(B \times N \times B)$, ce qui provoque une explosion combinatoire. Bien que son coût soit strictement identique à celui de notre Minimax actuel, l'Expectiminimax a l'inconvénient théorique de ne presque pas pouvoir être optimisé par des techniques d'élagage (comme l'Alpha-Beta). En effet, le calcul d'une espérance mathématique exige d'explorer toutes les probabilités jusqu'au bout, ce qui le coupe quasi systématiquement avant d'atteindre une profondeur de recherche de 2.

Performances

Sur le papier et dans notre code actuel, c'est la meilleure performance stratégique pour le Scrabble. L'évaluation du risque est mieux gérée : l'IA n'hésitera pas à ouvrir un "Mot Compte Triple" si elle calcule que la probabilité que l'adversaire possède les lettres exactes pour s'y placer est très faible. Le style de jeu devient plus naturel. Étant donné que son coût est identique à celui du Minimax dans notre code, l'Expectiminimax s'avère supérieur dans la prise de ses décisions.

5 Comparaison des deux Algorithmes

Les schémas ci-dessous mettent en évidence que la différence entre nos deux algorithmes n'est pas structurelle (l'arbre exploré est identique), mais réside uniquement dans la méthode d'agrégation des nœuds de simulation.

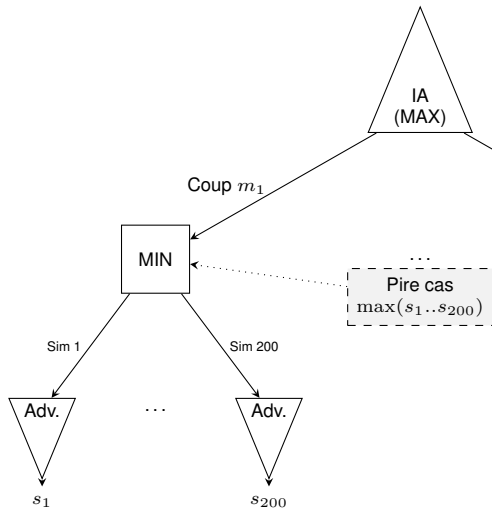


FIGURE 2 – Agrégation Pessimiste

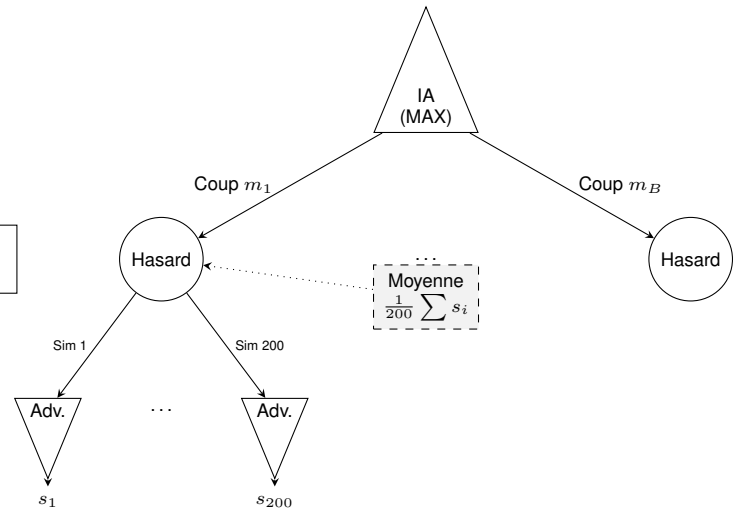


FIGURE 3 – Agrégation Probabiliste

Analyse des performances

Afin de confronter notre étude théorique à la réalité matérielle, nous avons instrumenté le code de notre solveur (`MinimaxSolver.java`) pour extraire des métriques physiques en conditions réelles de jeu. Le chronomètre d'interruption (`timeLimitMs`) était fixé à 5000 millisecondes.

Le tableau ci-dessous synthétise des échantillons représentatifs de la prise de décision de l'IA au cours d'une partie :

Algorithme	Temps	Statut	Coups IA Évalués	Simulations Adv.	Moy. Sim/Coup
Expectiminimax	5000 ms	Timeout	48 sur 84 (57%)	9 497	197
Expectiminimax	3575 ms	Complet	33 sur 33 (100%)	6 600	200
Minimax MC	5001 ms	Timeout	24 sur 58 (41%)	4 680	195
Minimax MC	5001 ms	Timeout	17 sur 41 (41%)	3 294	193
Minimax MC	2801 ms	Complet	10 sur 10 (100%)	2 000	200

TABLE 1 – Mesures physiques des algorithmes (Profondeur 2, SAMPLES_COUNT = 200)

Interprétation des résultats

L'analyse de ces logs d'exécution met en évidence trois points cruciaux concernant les limites de notre implémentation :

- **La limite du facteur de branchement** : Les données prouvent que la capacité de l'IA à évaluer l'arbre complet dépend entièrement de la complexité du plateau à l'instant T. Lorsque le nombre de mots possibles est faible (ex : 10 ou 33 coups), le moteur termine son parcours de Profondeur 2 confortablement avant le temps limite (2.8s et 3.5s). À l'inverse, dès que les choix s'élargissent (41, 58 ou 84 coups), l'IA est systématiquement bridée.
- **Les conséquences du coût élevé** : Dans le pire scénario mesuré (17 coups évalués sur 41), le *Timeout* force l'IA à jouer en ignorant totalement 59% des opportunités légales présentes sur le plateau, l'empêchant potentiellement de trouver le mot optimal.
- **L'efficacité de la limite de temps** : Le moteur est capable de simuler un volume massif de plateaux adverses (jusqu'à 9 497 en 5 secondes). Lorsque le *Timeout* est déclenché, on observe que la moyenne des simulations tombe légèrement sous la barre des 200 (192, 195, 197). Cela démontre que notre algorithme d'interruption fonctionne avec précision : il avorte la boucle interne de Monte-Carlo au milieu de l'évaluation du N -ième coup pour garantir la fluidité du jeu en temps réel.

Ces mesures empiriques confirment de manière irréfutable notre conclusion : bien que la Profondeur 2 soit mathématiquement atteinte, l'absence d'élagage dans nos approches de Monte-Carlo bride physiquement la rentabilité de l'arbre dès que le facteur de branchement dépasse la trentaine de mots candidats.

6 Le Machine Learning :

Contrairement aux algorithmes de recherche arborescente, le Machine Learning propose une approche statistique. Notre implémentation utilise un réseau de neurones conçu avec TensorFlow. L'architecture du modèle traite la recherche de mots comme un problème de classification multiclasse :

- **L'entrée** : Le chevalet du joueur est converti en un tenseur de 26 dimensions, représentant la fréquence de chaque lettre (de A à Z).
- **Les couches cachées** : Deux couches denses de 256 et 512 neurones analysent les motifs et anagrammes.
- **La sortie** : Une couche Softmax fournit une distribution de probabilités sur l'ensemble du dictionnaire (une probabilité par mot existant).

En jeu, l'agent Java interroge ce modèle pré-entraîné, filtre les mots ayant une probabilité de faisabilité supérieure à 0,1%, et trie les 100 meilleures prédictions.

Complexité et coût

La complexité est divisée en deux parties. La phase d'entraînement a une complexité linéaire dépendante de la taille du dictionnaire, du nombre d'époques et de la taille des lots. En partie, la complexité d'inférence est constante, soit $O(1)$. Le coût en jeu est faible (quelques millisecondes pour une matrice tensorielle). En revanche, le coût en mémoire vive est élevé en raison du chargement natif des bibliothèques TensorFlow et du graphe du modèle dans l'environnement Java.

Performances

La performance pure est très efficace pour trouver des anagrammes complexes. Mais la performance de jeu est nulle de manière autonome : le réseau actuel ignore totalement l'état du plateau et les contraintes de placement. Le moteur doit d'ailleurs prévoir une redirection vers les algorithmes arborescents si aucun des mots prédits par l'IA ne peut être légalement posé.

Conclusion :

La comparaison de ces méthodes met en évidence qu'aucun algorithme n'est parfait s'il est utilisé de manière isolée. L'Expectiminimax est le champion de la stratégie probabiliste, mais son coût le rend peu intéressant pour des recherches profondes. Le Minimax classique est efficace, surtout avec une bonne heuristique, mais excessivement défensif face à l'inconnu, et notre implémentation le prive de son avantage temporel théorique. Le Machine Learning est un très bon calculateur d'anagrammes, mais il est "aveugle" aux règles de placement sur le plateau.

La solution la plus efficace réside dans la fusion de ces approches. La complexité de l'Expectiminimax provient de son facteur de branchement immense (générer tous les mots possibles). En utilisant le Machine Learning comme un outil d'élagage probabiliste, le modèle TensorFlow peut instantanément suggérer les 20 meilleurs mots possibles depuis le chevalet. L'Expectiminimax n'a plus qu'à évaluer ces 20 branches réduites face aux risques du plateau, obtenant ainsi la certitude mathématique et probabiliste tout en respectant un temps de calcul extrêmement restreint.

7 DAWG et GADDAG

1. Introduction et contexte du problème

La génération de coups légaux au Scrabble impose une contrainte forte : la structure doit, en temps réel, trouver tous les mots jouables à partir d'une lettre déjà posée sur le plateau, que ce soit vers la gauche, la droite, ou les deux. Les structures classiques comme les tableaux de chaînes ou les arbres préfixes naïfs (Trie) sont inadaptées à cette contrainte, soit par leur lenteur de parcours, soit par leur consommation mémoire excessive face à un dictionnaire de 180 000 mots comme ceux que nous avons utilisés.

Deux structures de données académiques répondent à ce problème : le DAWG (Directed Acyclic Word Graph), conçu pour la validation rapide de mots, et le GADDAG, une extension inventée spécifiquement pour le Scrabble par Steven Gordon en 1994 permettant une recherche bidirectionnelle depuis n'importe quelle ancre. Afin de justifier notre choix final, nous avons implémenté et benchmarké les deux structures sur notre dictionnaire.

2. Le DAWG — Directed Acyclic Word Graph

2.1 Principe général

La structure de données DAWG représente une solution performante alliant vitesse d'exécution et économie de mémoire. Il s'agit d'un automate fini déterministe où chaque chemin partant de la racine correspond à un mot unique du dictionnaire. Sa particularité, qui la distingue d'un arbre préfixe classique, réside dans son optimisation poussée : le DAWG regroupe les mots partageant les mêmes préfixes, mais aussi les mêmes suffixes.

Par exemple, les mots CHAT et PLAT possèdent des débuts différents mais une terminaison identique ; lors de la construction, la structure va donc fusionner ces deux entrées au niveau de leur suffixe commun AT.

Cette approche permet de réduire drastiquement la redondance de l'information tout en garantissant une validation très rapide.

Lexicon:

car
cars
cat
cats
do
dog
dogs
done
ear
ears
eat
eats

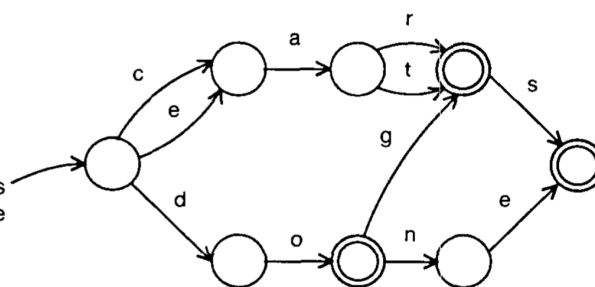


FIGURE 4 – Exemple d'une structure DAWG. (Source : [?])

2.2 Construction et complexité

La création d'un DAWG optimisé repose sur l'algorithme de Daciuk [?], qui impose une création des mots selon l'ordre lexicographique. Cette contrainte permet de traiter le dictionnaire de manière séquentielle : lorsqu'un nouveau mot est ajouté, l'algorithme identifie la partie de la structure qui ne sera plus modifiée et procède à la fusion des états équivalents, c'est-à-dire des nœuds possédant les mêmes transitions vers les mêmes états terminaux.

En matière de performance, la construction affiche une complexité temporelle de $O(|W| \times L)$, où $|W|$ représente le nombre total de mots et L leur longueur moyenne.

Pour la phase de jeu, la recherche d'un mot s'effectue en un temps linéaire $O(N)$ par rapport à la longueur N de la chaîne de caractères, ce qui garantit une validation quasi instantanée. Cependant, cette structure unidirectionnelle montre ses limites dès lors qu'une lettre « ancre » est déjà fixée au milieu d'un mot potentiel sur le plateau. Le DAWG ne permettant pas de remonter le chemin vers la racine pour identifier les débuts de mots possibles, le moteur de jeu est alors contraint d'explorer aveuglément tous les préfixes imaginables avant d'atteindre l'ancre, ce qui rend la génération de coups particulièrement inefficace dans ces configurations.

2.3 Avantages et limites pour le Scrabble

L'intérêt majeur du DAWG réside dans sa compacité exceptionnelle, permettant de compresser les centaines de milliers de mots d'un dictionnaire en un volume de données souvent inférieur à deux mégaoctets. Cependant, si cette structure est excellente pour la validation d'un mot saisi par un utilisateur, elle s'avère plus limitée pour la phase de génération automatique de coups. En raison de son parcours strictement unidirectionnel, de la racine vers les feuilles, le DAWG ne peut pas explorer efficacement les possibilités de jeu s'articulant autour d'une ancre déjà présente sur la grille.

Pour placer un mot contenant une lettre fixe en son centre ou à sa fin, l'algorithme est incapable de remonter le graphe pour identifier les préfixes compatibles. Il est alors contraint de tester à l'aveugle une multitude de combinaisons de lettres en espérant atteindre l'ancre, ce qui provoque une inefficacité notoire face à l'explosion combinatoire des tirages. Cette incapacité à traiter nativement la recherche bidirectionnelle souligne la nécessité d'une structure plus évoluée, capable de s'affranchir de la contrainte du parcours de gauche à droite : le GADDAG.

3. Le GADDAG — la solution pour le Scrabble

3.1 Principe et transformation

Le GADDAG, conçu par Steven Gordon en 1994, a donc apporté une réponse définitive au problème de l’ancrage en permettant une recherche bidirectionnelle efficace. Contrairement au DAWG qui ne stocke qu’une version linéaire de chaque mot, le GADDAG génère n représentations différentes pour un mot de longueur n , une pour chaque position possible d’une lettre sur le plateau. La structure repose sur une transformation : pour chaque lettre choisie comme ancre, la partie située à sa gauche est inversée, puis séparée de la partie droite par un caractère spécial « > » servant de délimiteur.

À titre d’exemple, le mot « CHAT » est décomposé en quatre représentations distinctes :

1. **C > H A T** (l’ancre est C)
2. **H C > A T** (l’ancre est H : on place C à gauche, puis on complète A-T à droite)
3. **A H C > T** (l’ancre est A : on place C et H à gauche, puis T à droite)
4. **T A H C >** (l’ancre est T : on complète tout le mot vers la gauche)

Cette méthode se généralise sans perte d’efficacité pour des mots plus complexes comme « SCRABBLE », qui produira huit entrées allant de « S>CRABBLE » jusqu’à la forme totalement inversée « ELBBARCS> ». Cette multiplication des chemins garantit que, pour n’importe quelle lettre du mot déjà présente sur la grille, il existe un chemin unique dans l’automate commençant par celle-ci et permettant de créer le reste du mot.

3.2 Semi-minimisation

L’augmentation du nombre de représentations par mot pourrait laisser craindre une explosion de la consommation mémoire, mais le GADDAG compense ce volume par une technique dite de semi-minimisation. Gordon a démontré que si les préfixes inversés divergent souvent, les suffixes situés après le caractère pivot « > » présentent une redondance massive. La semi-minimisation consiste donc à fusionner tous les états équivalents rencontrés après le délimiteur, exactement comme dans la construction d’un DAWG.

Pour le mot « CARE », les formes « AC>RE » et « RA>CE » voient leurs branches fusionnées avec n’importe quel autre mot du dictionnaire se terminant par « RE » ou « CE ». De même, des mots partageant des suffixes verbaux, comme « MARCHER » et « JOUER », partageront une structure commune pour leurs formes pivotées au-delà du délimiteur (par exemple « RAHC>MER » et « UOJ>ER » où la terminaison « ER » est mutualisée). Cette optimisation rend la structure déterministe et permet de conserver un seul chemin par couple (mot, ancre), tout en maintenant une empreinte mémoire tout à fait acceptable pour les systèmes modernes.

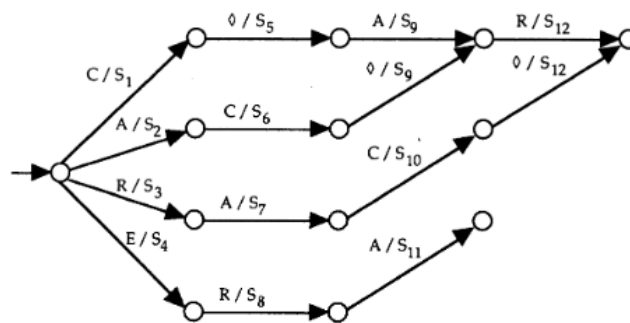


FIGURE 5 – Graphe du GADDAG semi-minimisé pour « CARE ». (Source : [?])

3.3 Utilisation dans le moteur de jeu

L’intégration du GADDAG au sein du moteur de jeu transforme radicalement la génération de coups en la rendant purement itérative. Pour chaque case « ancre » identifiée sur le plateau, l’algorithme entame un parcours du graphe en deux phases distinctes. Dans un premier temps, il explore les branches correspondant à la partie gauche du mot (le préfixe inversé) en remontant vers les cases vides à gauche ou en haut de l’ancre. Dès qu’il rencontre le caractère « > » dans l’automate, il bascule dans la seconde phase pour compléter le mot vers la droite ou vers le bas

avec les lettres restantes du tirage.

Cette approche permet de découvrir l'intégralité des coups légaux en un seul passage dans l'automate, sans jamais avoir besoin de revenir en arrière ou de deviner une position de départ. En éliminant les tests de préfixes inutiles propres au DAWG, le GADDAG permet à l'IA d'évaluer des milliers de possibilités par seconde, garantissant une réactivité optimale même face à des dictionnaires extensifs et des contraintes de temps réelles.

4. Analyse du benchmark et justification du choix

TABLE 2 – Benchmark DAWG vs GADDAG — Lexique : 178 691 mots

		Structure	Temps de chargement		
		DAWG	624 ms		
		GADDAG	1 443 ms		

Mot	Existe	DAWG	GADDAG
APPLE	vrai	3,79 μ s	8,91 μ s
ZYZZYVA	vrai	4,32 μ s	9,70 μ s
SCRABBLE	vrai	3,96 μ s	8,08 μ s
JAVA	vrai	2,75 μ s	8,20 μ s
INEXISTANT	faux	4,19 μ s	13,94 μ s

Ancre	Rack	DAWG	GADDAG	Coups (D)	Coups (G)
E	RSTLNA	2,966 ms	4,752 ms	170	170
S	CRABBLE	1,184 ms	1,288 ms	112	112
Z	AEIOUST	0,467 ms	0,269 ms	16	16
X	AILNOR	0,289 ms	0,178 ms	11	11

Les résultats du benchmark, conduits sur un lexique de 178 691 mots, révèlent une image nuancée qui justifie pleinement le choix du GADDAG comme structure centrale du moteur de jeu.

Temps de chargement et validation simple. Le GADDAG accuse un temps de chargement plus élevé (1 443 ms contre 624 ms pour le DAWG), ce qui s'explique directement par sa transformation structurelle : chaque mot de longueur n génère n représentations distinctes, alourdissant mécaniquement la phase de construction. De même, sur la validation isolée d'un mot (*contains*), le DAWG reste environ deux fois plus rapide (3 à 4 μ s contre 8 à 14 μ s). Ces écarts sont cependant sans conséquence pratique : ces opérations sont réalisées une seule fois à l'initialisation ou de manière ponctuelle, et les valeurs absolues restent négligeables à l'échelle d'une partie.

Génération de coups : l'avantage décisif du GADDAG. C'est sur la génération de coups que le choix se justifie pleinement. Les résultats montrent une tendance claire : plus l'espace de jeu est contraint (peu de mots possibles), plus le GADDAG surpasse le DAWG. Pour l'ancre **Z** avec le rack AEIOUST (16 coups trouvés), le GADDAG est **42 % plus rapide** (0,269 ms contre 0,467 ms) ; pour **X** avec AILNOR (11 coups), il est **38 % plus rapide** (0,178 ms contre 0,289 ms). Ces cas correspondent précisément aux situations les plus fréquentes en fin de partie ou sur des lettres rares, où l'espace combinatoire est restreint mais où chaque coup compte. En fin de partie, le plateau étant déjà largement occupé, les ancrs disponibles sont nombreuses mais les extensions possibles réduites : le GADDAG, en naviguant directement depuis chaque ancre sans exploration aveugle des préfixes, évite une explosion combinatoire inutile et identifie les rares coups légaux restants bien plus efficacement que le DAWG, qui continuerait à tester des chemins stériles.

Sur les racks plus généreux (ancre **E** avec RSTLNA, 170 coups), le DAWG reprend l'avantage. Cela s'explique par

le fait que dans ce cas, la quantité de chemins à explorer est si importante que le surcoût structurel du GADDAG se ressent. Toutefois, les deux structures trouvent **exactement le même nombre de coups** dans tous les cas testés, ce qui confirme la correction de l'implémentation.

Conclusion du choix. Le DAWG est une structure excellente pour la validation de mots, mais son parcours strictement unidirectionnel le contraint à explorer aveuglément des préfixes inutiles dès qu'une ancre est présente sur la grille. Le GADDAG, en encodant nativement toutes les positions d'ancrage possibles, élimine ce surcoût algorithmique et offre des performances supérieures précisément là où cela importe le plus : les situations tendues, typiques d'une vraie partie de Scrabble. Le léger surcoût en mémoire et en temps de chargement est un compromis largement acceptable au regard du gain en réactivité durant le jeu.

Première partie

Spécifications Étendues

8 F21 — Sauvegarde et restauration d'une partie

Description

Le programme doit pouvoir sérialiser une partie en cours dans un fichier texte ASCII et reconstruire une partie complète à partir d'un fichier de sauvegarde. Le parseur doit être robuste aux erreurs de format et les signaler avec précision (type d'erreur et numéro de ligne). L'implémentation repose sur les classes `SaveManager.java` et `GameLoader.java`.

Prérequis

- **F2** — **Configuration** : Paramètres par défaut (langue, blitz, etc.) via `.scrabblerc` — `ConfigLoader.java`
- **F10** — **Règles du jeu** : État complet du jeu : plateau, scores, joueurs, historique, tour courant
- **F15** — **Shell interactif** : Commandes `save FILE` et `load FILE` — `GameControllerAux.java`
- **F22** — **Commentaires** : Gestion des commentaires `#` et blocs `{ }` — `GameLoader.java`
- **F23/F24/F25** — **Formats** : Sections `[settings]`, `[game]` et `[history]` du fichier de sauvegarde

Type de besoin

Fonctionnel.

Sous besoins 1. Sauvegarder la partie dans un fichier

Description : Sérialiser l'état courant du jeu dans un fichier texte ASCII en écrivant successivement les trois sections obligatoires.

Fonction associée :

```
void saveGame(Game game, String filePath) throws IOException
```

Actions :

- Ouvrir (ou créer) le fichier au chemin indiqué.
- Écrire successivement `[settings]`, `[game]` puis `[history]`.
- Fermer le fichier proprement.
- Remonter `IOException` en cas d'échec (gérée par CLI ou GUI).

Résultat attendu :

- *Succès* : fichier complet et bien formé écrit sur le disque.
- *Échec* : exception `IOException` remontée, aucun fichier partiel conservé.

Test unitaire :

- **Cas 1** : Entrée : état complexe (plateau avec mots, 2 joueurs, scores, racks, coups pass/play). Attendu : fichier créé avec 3 sections, vérifié via `SaveManagerTest.java`.
- **Cas 2** : Entrée : conversion de coordonnées `a1v / h8h`. Attendu : format des coordonnées correct dans `[history]`.
- **Cas 3** : Entrée : partie vide (aucun coup joué). Attendu : plateau serialisé en lignes de tirets, section `[history]` vide.

Sous besoins 2. Serialiser la section [settings]

Description : Ecrire les parametres globaux de la partie et les parametres IA joueur par joueur sous la section [settings].

Parametres écrits :

- blitz — mode blitz active ou non.
- super-scrabble — plateau 21×21.
- players-count — nombre de joueurs.
- debug / verbose — niveaux de trace.
- player-N-type — human ou ai.
- player-N-ai-mode — algorithme IA (ex.: minimax).
- player-N-name — nom du joueur.

Resultat attendu :

- Section [settings] complete et coherente avec la configuration de partie.

Sous besoins 3. Serialiser la section [game]

Description : Ecrire l'état du plateau, les racks et les scores de chaque joueur.

Fonctions internes :

```
saveBoard(PrintWriter writer, Board board)
serializeRack(Player p)
```

Format produit :

- Premiere ligne : index du joueur courant (1-based).
- Plateau ligne par ligne : lettre majuscule ou tiret - pour une case vide.
- rack-N: LETTRES — reserve de lettres du joueur N.
- score-N: valeur — score courant du joueur N.
- language en — langue de la partie (non dans [settings]).

Test unitaire :

- **Cas 1 :** Entree : plateau avec mot COUNT en g5h. Attendu : la ligne g contient C O U N T aux bonnes colonnes.
- **Cas 2 :** Entree : plateau entierement vide. Attendu : toutes les cases sont representees par -.
- **Cas 3 :** Entree : joueur 1 score 15 / joueur 2 score 10. Attendu : presence de score-1: 15 et score-2: 10.

Sous besoins 4. Serialiser la section [history]

Description : Ecrire les coups joues dans l'ordre chronologique au format defini par F25.

Fonctions internes :

```
saveHistory(PrintWriter writer, Game game, List<Move> history)
readFullWordFromBoard(Board board, Move move)
convertPointToCoord(Point p)
```

Formats de coup supportes :

- N pass — le joueur passe son tour.
- N exchange ABC — le joueur echange des lettres.
- N g5h WORD ou N e6v WORD — coup normal horizontal ou vertical.

Test unitaire :

- **Cas 1 :** Entree : joueur 1 joue COUNT en g5h, joueur 2 joue PHOTO en e6v. Attendu : presence de 1 g5h COUNT et 2 e6v PHoTO en tete d'historique.
- **Cas 2 :** Entree : aucun coup joue (partie neuve). Attendu : section [history] presente, sans ligne de coup.
- **Cas 3 :** Entree : coup de type pass. Attendu : format N pass ecrit correctement.
- **Cas 4 :** Entree : coup de type exchange. Attendu : format N exchange LETTRES ecrit correctement.

Sous besoins 5. Charger une partie depuis un fichier

Description : Parser le fichier de sauvegarde et reconstruire un objet `Game` complet (joueurs, plateau, racks, scores, tour courant et historique).

Fonction associee :

`Game loadGame(String filePath) throws Exception`

Actions :

- Prelecture du fichier pour detecter le mode super-scrabble et creer le bon plateau (15×15 ou 21×21).
- Lecture sequentielle et parsing des sections `[settings]`, `[game]` et `[history]`.
- Reconstruction des joueurs humains et IA avec leurs parametres.
- Reconstruction de l'historique complet des coups.

Gestion d'erreurs :

- Toute erreur de parsing est remontee sous la forme `Format error at line X: <detail>`.
- La CLI et la GUI affichent ensuite un message utilisateur lisible.
- L'etat courant du jeu n'est jamais ecrase en cas d'echec.

Test unitaire :

- **Cas 1 :** Entree : fichier valide et complet. Attendu : `Game` non nul avec plateau, scores et historique coherents (`GameLoaderTest.java`).
- **Cas 2 :** Entree : plateau 21×21 (super-scrabble). Attendu : detection correcte et creation du bon plateau.
- **Cas 3 :** Entree : fichier contenant `score-1: abc`. Attendu : exception `Format error at line X: ...`
- **Cas 4 :** Entree : section `[game]` manquante. Attendu : exception avec message explicite.
- **Cas 5 :** Entree : coup `exchange` dans l'historique. Attendu : coup restaure correctement.
- **Cas 6 :** Entree : fichier inexistant. Attendu : exception fichier introuvable.

Sous besoins 6. Parser et ignorer les commentaires

Description : Supprimer les commentaires avant le parsing tout en conservant une numerotation de ligne fiable pour les messages d'erreur, conformement a F22.

Fonction interne :

`processLineComments(String line)`

Comportement :

- Supprime tout ce qui suit un `#` sur une ligne (commentaire en ligne).
- Ignore les blocs `{ ... }` potentiellement multi-lignes via un etat `isInBlockComment`.
- Continue le parsing ligne par ligne en maintenant le compteur de lignes.

Test unitaire :

- **Cas 1 :** Entree : `debug=true # activer le debug`. Attendu : `debug=true` apres suppression du suffixe commente.
- **Cas 2 :** Entree : bloc `{ commentaire\nsur 2 lignes }` puis `language=fr`. Attendu : bloc supprime, `language=fr` conserve.
- **Cas 3 :** Entree : fichier sans commentaire. Attendu : contenu strictement identique en sortie.

Intégration dans l'architecture du projet

Scénario utilisateur

1. L'utilisateur joue plusieurs coups en CLI ou via l'interface graphique.
2. En CLI, il tape : `save ma_partie.scrabble`
3. `SaveManager` écrit `[settings]`, `[game]` puis `[history]` dans le fichier.
4. Le shell affiche un message de succès à l'utilisateur.
5. L'utilisateur relance le programme et tape : `load ma_partie.scrabble`
6. `GameLoader` reconstruit le `Game` : joueurs, plateau, tour courant et historique.
7. Si une ligne est invalide, le programme affiche `Format error at line X: ...` et n'écrase pas l'état courant.

Module	Responsabilité	Détail
savefiles	SaveManager.java GameLoader.java	SaveManager : sérialisation de la partie. GameLoader : parsing et reconstruction du Game.
CLI	GameControllerAux.java	save FILE → SaveManager.saveGame load FILE → GameLoader.loadGame Erreurs affichées via cliView.displayError
GUI	ScrabbleGui.java	Menu Save → SaveManager.saveGame Menu Load → GameLoader.loadGame Rebind du contrôleur et rafraîchissement UI après chargement.
Configuration F2	ConfigLoader.java	Lit .scrabblerc dans le HOME. Note : le chemin par défaut de save/load depuis .scrabblerc n'est pas encore pris en charge si FILE est absent.